ChemLink: Plataforma de Orquestación para Docking Molecular en Entornos HPC

Juan Felipe Santos Rodríguez, Samuel Matiz García, Camilo Andrés Navarro Navarro
Universidad del Norte
Barranquilla, Colombia
Docente: Augusto Salazar | Co-asesor: Daniel Romero
2026

I. Introducción

En las últimas décadas, la bioinformática y la química computacional se han consolidado como pilares fundamentales en el descubrimiento racional de fármacos y en la comprensión de mecanismos moleculares complejos. Estas disciplinas permiten modelar, simular y analizar interacciones biológicas a nivel atómico mediante herramientas computacionales avanzadas, reduciendo significativamente los costos y tiempos asociados a la experimentación exclusivamente experimental.

Entre las técnicas más relevantes se encuentran el *docking* molecular y la dinámica molecular. El *docking* molecular permite predecir la orientación y afinidad de unión entre una proteína y un ligando, mientras que la dinámica molecular posibilita evaluar la estabilidad estructural y el comportamiento temporal de complejos biomoleculares. La ejecución eficiente de estas técnicas requiere altos recursos computacionales, lo que ha impulsado el uso de entornos de Computación de Alto Rendimiento (HPC), capaces de distribuir cargas de trabajo y aprovechar aceleración mediante GPUs.

En este contexto, las tendencias actuales en investigación computacional apuntan hacia la automatización de *pipelines* científicos, la integración con gestores de colas como SLURM, el uso de arquitecturas modulares y la implementación de sistemas de almacenamiento centralizado que permitan manejar grandes volúmenes de datos de manera organizada y reproducible. La reproducibilidad científica, en particular, se ha convertido en un criterio esencial dentro de los estudios computacionales.

No obstante, en muchos entornos académicos y de investigación, la ejecución de experimentos de *docking* molecular aún depende de procesos manuales y de la integración *ad hoc* de diversas herramientas. Esta situación genera dificultades en la gestión de experimentos, limita la escalabilidad de los estudios y dificulta mantener una trazabilidad clara de los resultados obtenidos.

II. Planteamiento del Problema

En los entornos académicos donde se desarrollan estudios de *docking* molecular, la ejecución de experimentos computacionales suele depender de una integración manual y fragmentada de múltiples herramientas. Aunque existen soluciones robustas para el cálculo molecular, como AutoDock-GPU [1], y gestores de colas ampliamente adoptados como SLURM [2], su articulación operativa dentro de un flujo experimental coherente no se encuentra formalizada ni automatizada.

El problema no radica en la inexistencia de herramientas científicas eficientes, sino en la ausencia de un sistema integrador que permita orquestar, monitorear y estructurar su ejecución bajo principios de automatización, control y reproducibilidad académica.

Actualmente, los investigadores deben preparar manualmente *scripts* de envío al clúster, configurar parámetros de ejecución, administrar rutas de almacenamiento distribuidas y monitorear el estado de los trabajos mediante comandos de bajo nivel. La integración entre herramientas de cálculo, gestión de recursos HPC y almacenamiento compartido no responde a una arquitectura estructurada, sino a prácticas individuales que varían entre usuarios.

Esta fragmentación técnica genera múltiples efectos negativos:

- Retrasos en los ciclos de experimentación debido a errores de configuración y reenvío de tareas.
- Pérdida de reproducibilidad al no existir trazabilidad formal de parámetros, versiones y resultados.
- Sobrecarga técnica en los investigadores, quienes deben dedicar tiempo significativo a tareas operativas en lugar de análisis científico.
- Dificultad para escalar experimentos cuando se requiere ejecutar múltiples simulaciones en paralelo dentro del clúster.

Como consecuencia, el proceso de investigación computacional se vuelve menos eficiente, más propenso a inconsistencias y dependiente del conocimiento técnico individual, lo que limita el aprovechamiento óptimo de la infraestructura HPC disponible.

III. Justificación

El desarrollo de herramientas que faciliten la gestión de experimentos computacionales se ha vuelto fundamental para aprovechar adecuadamente las infraestructuras de computación de alto rendimiento disponibles en entornos académicos. En el caso del *docking* molecular, la ausencia de sistemas que integren de manera estructurada la ejecución de tareas, la gestión de recursos y el almacenamiento de resultados dificulta la realización de estudios eficientes y reproducibles.

Ante esta situación, resulta pertinente diseñar una plataforma que permita automatizar y estructurar los *pipelines* de *docking* molecular en entornos HPC. Una solución de este tipo contribuiría a optimizar el uso del clúster de cómputo, reducir la carga operativa sobre los investigadores y mejorar la trazabilidad de los experimentos mediante una gestión organizada de parámetros y resultados.

De esta manera, el proyecto busca aportar una herramienta que facilite la ejecución de estudios computacionales más eficientes, escalables y reproducibles dentro del ámbito académico.

IV. Restricciones y Supuestos de Diseño

El desarrollo del sistema considera algunas restricciones técnicas y operativas que pueden influir en su implementación:

- En primer lugar, el proyecto depende de herramientas externas especializadas como AutoDock-GPU, las cuales deben estar correctamente instaladas y configuradas en el entorno de ejecución para garantizar el funcionamiento del proceso de simulación molecular.
- Asimismo, el sistema requiere una configuración adecuada de permisos y del entorno de ejecución dentro del clúster o infraestructura de cómputo, lo que implica que los usuarios tengan acceso autorizado a los recursos necesarios como GPU, almacenamiento y colas de procesamiento.
- Otra restricción importante es la posible sobrecarga de las GPU si no se gestiona correctamente la distribución de tareas. Por esta razón, el diseño contempla mecanismos de control para evitar la saturación de recursos y asegurar un uso eficiente del *hardware* disponible.
- Finalmente, se asume que el tiempo disponible para realizar pruebas en el entorno real será limitado, por lo que parte de la validación inicial del sistema deberá realizarse en entornos de prueba o simulación antes de su ejecución completa en la infraestructura definitiva.

V. Objetivos

- **Diseñar** una arquitectura de software modular y desacoplada, organizada en capas (CLI, Core, Adapters y Utils), que permita estructurar claramente las responsabilidades del sistema, facilite su mantenibilidad y habilite futuras extensiones dentro del ecosistema HPC del laboratorio.
- **Automatizar** el *pipeline* completo de *docking* molecular, integrando la detección automática de sitios de unión, la generación dinámica de cajas de *docking*, la preparación y validación de archivos de entrada, la ejecución paralela del *docking* con GPU y la verificación estructurada de los resultados generados.
- **Implementar** un sistema de ejecución paralela híbrido que permita tanto el multiprocesamiento local como la generación automática de *scripts* para la gestión eficiente de recursos en el clúster.
- **Desarrollar** mecanismos de validación y control de recursos computacionales que evalúen la disponibilidad de CPU, memoria y GPU antes y durante la ejecución de experimentos, con el propósito de prevenir sobrecargas, optimizar la asignación de tareas y garantizar la estabilidad operativa del clúster.
- **Incorporar** strategies básicas de tolerancia a fallos, incluyendo control de *timeouts*, detección de ejecuciones incompletas o fallidas y registro estructurado de errores, asegurando la robustez del sistema ante eventos inesperados.
- **Diseñar e implementar** un módulo de análisis estructurado de resultados que permita extraer automáticamente métricas relevantes como afinidad y RMSD, generar rankings globales de ligandos, calcular estadísticas descriptivas y exportar reportes reproducibles que faciliten la trazabilidad científica.
- **Evaluar** el desempeño del sistema mediante la comparación entre ejecución secuencial y distribuida, midiendo métricas como tiempo total de ejecución, *speedup* paralelo, eficiencia computacional y tasa de éxito experimental, con el fin de validar el impacto de la solución en el entorno HPC del laboratorio.

VI. Alcance

El proyecto consiste en diseñar e implementar una plataforma científica basada en línea de comandos (CLI), modular y orientada a entornos de cómputo de alto rendimiento (HPC), que permita automatizar y optimizar la ejecución de experimentos de *docking* molecular y dinámica molecular dentro de un clúster distribuido. La plataforma integrará herramientas especializadas, gestión eficiente de recursos computacionales (CPU y GPU) y mecanismos de paralelización para mejorar el rendimiento en la ejecución de simulaciones.

Asimismo, el sistema permitirá la organización y trazabilidad de los experimentos realizados, incluyendo la validación básica de resultados y la generación automática de reportes reproducibles. Con esto se busca mejorar la eficiencia computacional, facilitar la gestión de experimentos científicos y proporcionar una base escalable para el desarrollo del ecosistema Chemlab.

VII. Estado del Arte y Soluciones Relacionadas

VII-A. Herramientas de docking molecular

En el ámbito del *docking* molecular, AutoDock [3] implementa algoritmos basados en búsquedas estocásticas y funciones de puntuación energéticas para estimar afinidad proteína–ligando. Su versión acelerada, AutoDock-GPU [1], optimiza el cálculo mediante paralelización masiva en GPU, reduciendo significativamente los tiempos de ejecución en entornos de alto rendimiento. Sin embargo, estas herramientas están orientadas principalmente al cálculo molecular en sí mismo y no incluyen mecanismos nativos de orquestación distribuida, trazabilidad estructurada de experimentos ni integración directa con sistemas de almacenamiento centralizado.

VII-B. Herramientas de dinámica molecular

Para simulaciones de dinámica molecular, GROMACS [4] ofrece paralelización híbrida (MPI + OpenMP), escalabilidad

en *clusters* y optimización para arquitecturas CPU/GPU. Aunque es altamente eficiente desde el punto de vista computacional, su ejecución en entornos HPC requiere configuración manual de *scripts*, definición explícita de recursos y manejo independiente de resultados, lo que incrementa la complejidad operativa.

VII-C. Gestión de recursos en entornos HPC

En la capa de infraestructura, SLURM [2] es uno de los gestores de colas más utilizados en *clusters* académicos. Permite asignación dinámica de recursos, control de prioridades y monitoreo del estado de los *jobs*. No obstante, SLURM opera principalmente a nivel de administración de recursos y no provee abstracciones orientadas a experimentos científicos. Su interacción se basa en *scripts* y comandos de bajo nivel como `sbatch`, `squeue` y `scancel`, lo que exige conocimientos técnicos específicos por parte de los usuarios.

VII-D. Monitoreo de infraestructura en entornos HPC

En cuanto al monitoreo de infraestructuras distribuidas, Prometheus [5] se ha consolidado como una de las herramientas más utilizadas para la recolección y análisis de métricas en sistemas y servicios desplegados en entornos de computación distribuida. Prometheus permite capturar métricas de rendimiento mediante un modelo de series temporales y ofrece un lenguaje de consulta especializado (PromQL) para analizar el estado de los recursos computacionales. Esta plataforma soporta la supervisión de múltiples nodos en infraestructuras como clusters HPC, permitiendo monitorear variables críticas como uso de CPU, memoria, almacenamiento y aceleradores GPU. No obstante, aunque proporciona capacidades robustas de observabilidad del sistema, Prometheus no está orientado a la orquestación de pipelines científicos ni a la gestión de experimentos de docking molecular, por lo que su funcionalidad se limita principalmente al monitoreo de la infraestructura y no a la automatización integral de flujos de trabajo científicos.

VII-E. Limitaciones del ecosistema actual

Desde una perspectiva arquitectónica, se observa que:

- Las herramientas de cálculo (AutoDock-GPU, GROMACS) se enfocan en procesamiento científico.
- Los gestores de recursos (SLURM) administran infraestructura.
- Las plataformas de monitoreo de infraestructura (Prometheus) permiten la recolección y análisis de métricas del sistema en entornos distribuidos.

Ninguno integra de manera nativa almacenamiento científico estructurado (NAS), monitoreo orientado a experimentos ni control de reproducibilidad específico para *docking* académico. Esta fragmentación implica que el investigador debe asumir la responsabilidad de integrar manualmente múltiples capas tecnológicas: preparación de datos, generación de *scripts*, envío al clúster, seguimiento de ejecución, recuperación de resultados y análisis posterior. El resultado es una arquitectura implícita, no formalizada, dependiente de prácticas individuales y vulnerable a inconsistencias.

Por tanto, el vacío identificado no corresponde a la ausencia de herramientas de cálculo eficientes, sino a la falta de una arquitectura integradora que articule computación científica, gestión HPC, almacenamiento centralizado y monitoreo experimental bajo un modelo coherente, modular y reproducible.

VIII. Diagramas de Arquitectura

Para ilustrar el diseño modular y desacoplado propuesto para ChemLink, se presenta a continuación el Modelo C4 de la plataforma. La Figura **??** detalla la estructura desde el contexto de alto nivel hasta la implementación a nivel de código.

IX. Requerimientos Preliminares

IX-A. Funcionales

- Generar automáticamente *scripts* de ejecución para SLURM.
- Enviar *jobs* al clúster de cómputo.
- Monitorear el estado de ejecución de los experimentos.
- Ejecutar herramientas científicas como AutoDock-GPU y Fpocket [6].
- Almacenar resultados de experimentos en el sistema NAS del laboratorio.
- Generar reportes estructurados a partir de los resultados obtenidos.

IX-B. No Funcionales

- El sistema no debe introducir una sobrecarga mayor al $5\,\%$ respecto a la ejecución manual de los experimentos.
- Disponibilidad del sistema $\geq 95\,\%$ durante las pruebas experimentales.
- Soporte para ejecución concurrente de múltiples experimentos.
- Garantizar trazabilidad completa de los experimentos, incluyendo parámetros, ejecuciones y resultados.
- Arquitectura modular y extensible que permita integrar nuevas herramientas científicas en el futuro.

X. Criterios de Aceptación Inicial

- El sistema ejecuta un *docking* completo con un solo comando.
- El *job* es enviado correctamente a SLURM.
- El estado puede ser consultado automáticamente.
- Los resultados se almacenan correctamente en NAS.
- Se genera un reporte estructurado al finalizar.
- No se requieren intervenciones manuales intermedias.

XI. Plan de Trabajo

En la Tabla I se detalla el cronograma estimado de actividades a desarrollar a lo largo del proyecto. El cronograma está organizado en cinco fases principales y distribuido a lo largo de 12 semanas (desde la S4 hasta la S15).

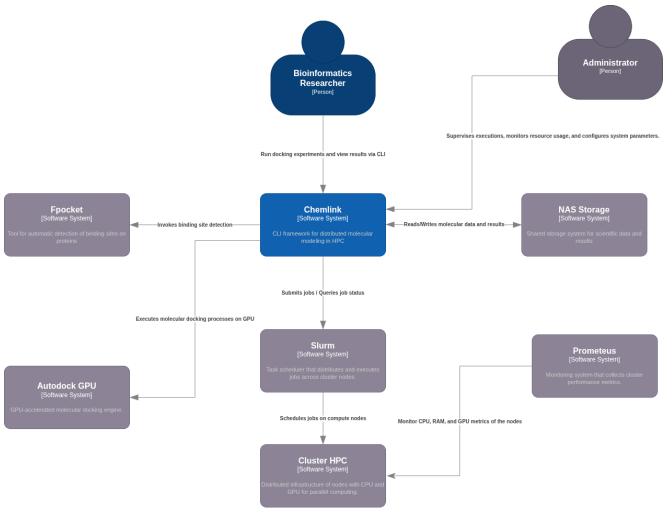

Figura 1: Nivel 1: Contexto

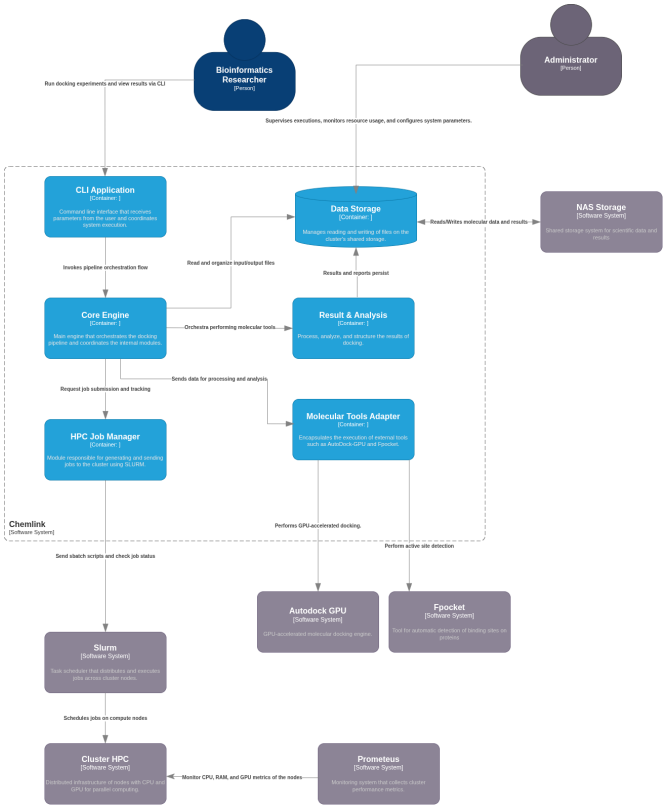

Figura 2: Nivel 2: Contenedores

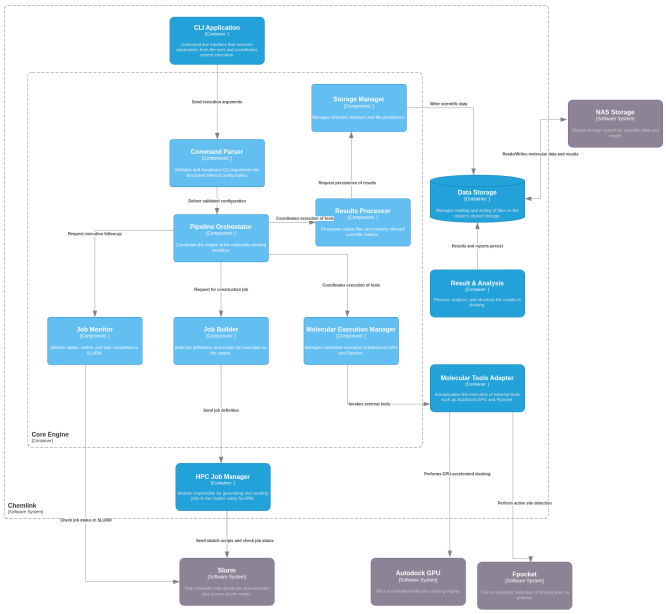

Figura 3: Nivel 3: Componentes

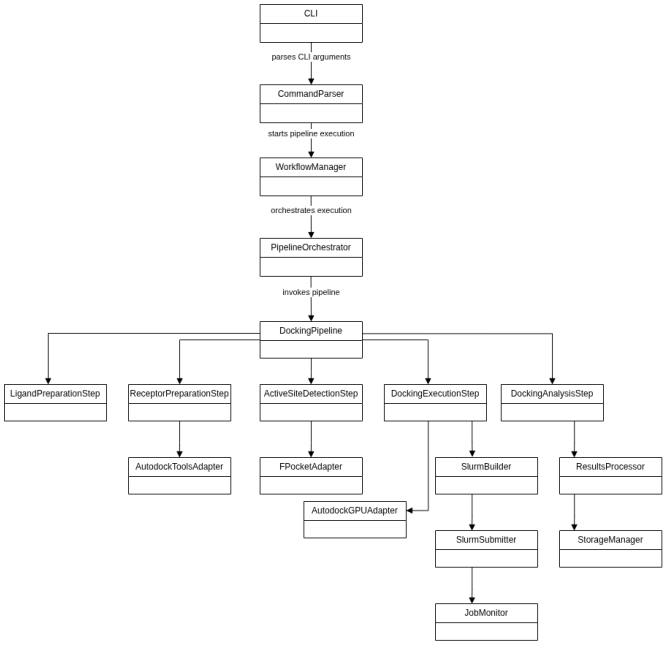

Figura 4: Nivel 4: Clases

Tabla I: Plan de Trabajo (Semanas 4 a 15)

Fase	Actividad	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
Fase 1 – Análisis y Especificación													
	Análisis del problema y entorno HPC	X	X										
	Requerimientos y métricas		X	X									
	Arquitectura de alto nivel			X	X								
Fase 2 – Diseño Detallado y Preparación													
	Diseño interno de módulos				X	X							
	Diseño paralelización híbrida					X	X						
	Validación entorno HPC y *baseline*						X	X					
Fase 3 – Implementación Modular													
	Implementación CLI							X	X				
	Implementación Core Pipeline								X	X			
	Integración con AutoDock-GPU									X	X		
	Paralelización multiproceso y *cluster*										X	X	
	Control de recursos y tolerancia a fallos											X	X
Fase 4 – Análisis y Reportes													
	Módulo de análisis (afinidad, RMSD, ranking)												X
	Generación de reportes reproducibles												X
Fase 5 – Evaluación y Consolidación													
	Evaluación de desempeño (*speedup*, eficiencia)												X
	Optimización final												X
	Integración total y preparación final												X

Nota: Las semanas indicadas en el cronograma (S4 a S15) corresponden a las semanas académicas del semestre en curso, y no a las semanas transcurridas desde el inicio del proyecto.

XII. Riesgos del Proyecto

El desarrollo de una plataforma científica orientada a entornos de cómputo de alto rendimiento implica ciertos riesgos técnicos y operativos:

- **Complejidad técnica:** El sistema debe integrar múltiples componentes como la generación de *scripts* para SLURM, la ejecución de herramientas científicas como AutoDock-GPU y la gestión de almacenamiento en NAS dentro de un entorno HPC distribuido. Esta integración puede presentar desafíos de estabilidad y coordinación de recursos.
- **Plazos de desarrollo:** Existe el riesgo de que la implementación y las pruebas rigurosas en un entorno de clúster real requieran más tiempo del previsto, retrasando la fase de evaluación.
- **Curva de aprendizaje:** La adopción inicial del sistema por parte de los investigadores puede verse afectada si la interfaz de línea de comandos (CLI) y los flujos de uso no se diseñan de forma suficientemente clara e intuitiva.

Referencias

[1] A. Santos-Martins *et al.*, "Accelerating AutoDock4 with GPUs and Gradient-Based Local Search," *Journal of Chemical Theory and Computation*, vol. 42, no. 2, pp. 1060-1073, 2021. [En línea]. Disponible: https://github.com/ccsb-scripps/AutoDock-GPU

[2] A. B. Yoo, M. A. Jette, y M. Grondona, "Slurm: Simple linux utility for resource management," en *Job Scheduling Strategies for Parallel Processing*, 2003, pp. 44–60. [En línea]. Disponible: https://slurm.schedmd.com/

[3] O. Trott y A. J. Olson, "AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading," *Journal of Computational Chemistry*, vol. 31, no. 2, pp. 455–461, 2010. [En línea]. Disponible: https://autodock.scripps.edu/

[4] M. J. Abraham *et al.*, "GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers," *SoftwareX*, vol. 1-2, pp. 19–25, 2015. [En línea]. Disponible: https://www.gromacs.org/

[5] B. Sigelman *et al.*, "Prometheus: A Next-Generation Monitoring System," in *Proc. IEEE/ACM Int. Conf. on Cloud Engineering (IC2E)*, Tempe, AZ, USA, 2015, pp. 27–32. [En línea]. Disponible: https://prometheus.io/

[6] V. Le Guilloux, P. Schmidtke, y P. Tuffery, "Fpocket: An open source platform for ligand pocket detection," *BMC Bioinformatics*, vol. 10, no. 168, 2009. [En línea]. Disponible: https://github.com/fpocket/fpocket